AdaptaBrasil · Mapa de Indicadores

Mapa do Brasil inteiro sem espera

Como o aplicativo de testes passou a exibir os 468 mil setores censitários do país em poucos segundos — e os mapas de estado e município sem falhas entre polígonos: tiles vetoriais gerados pelo PostGIS para todos os mapas, limites de estados e municípios, cores pré-calculadas por indicador e, no zoom do país, municípios coloridos pela média dos seus setores.

Repositório: github.com/AdaptaBrasil/webtests · Novembro de 2026

Resumo

Originalmente, um mapa de estado enviava ao navegador todos os polígonos (GeoJSON) e um valor por polígono. Para o país inteiro isso não funciona: são **468.097 setores censitários** com **1,2 GB de geometria**, e ler um indicador para o Brasil todo exige percorrer a tabela `value` inteira (~200 s). A solução separa o problema em duas partes, ambas calculadas uma única vez e guardadas no banco:

- **Geometria em tiles vetoriais** (PostGIS `ST_AsMVT`): o navegador baixa só os pedaços do mapa que estão na tela, já simplificados para o nível de zoom.
- **Cores como uma "string de classes"** por indicador: um caractere por setor com o índice da faixa da legenda — cerca de **8 KB** comprimidos para o país inteiro.

Depois, a mesma camada de tiles passou a ser usada também nos mapas de **estado e município** (que agora só baixam os valores), e uma segunda camada de tiles desenha os **limites de estados e municípios**. Isso eliminou as falhas ("slivers") entre setores vizinhos e reduziu a carga de Minas Gerais de 45,5 s para cerca de 3 s.

Por fim, no zoom do país (abaixo do zoom 8) o mapa mostra os **5.570 municípios**, cada um com a **média dos valores dos seus setores**, em vez de setores menores que um pixel. As cidades passaram a aparecer e sumiram os pontinhos cinza de setores grandes sem dado (seção 7).

> **Resultado:** escolhendo **Brasil** no campo Estado, o mapa nacional aparece em cerca de 5–8 segundos (a maior parte é o navegador aplicando as cores); trocar de indicador custa um download de 8 KB; aproximar o zoom carrega apenas os tiles da área visível. Um estado grande como MG (51 mil setores) abre em 3–4 s; um município como Belo Horizonte, em 0,6 s.

1. O problema

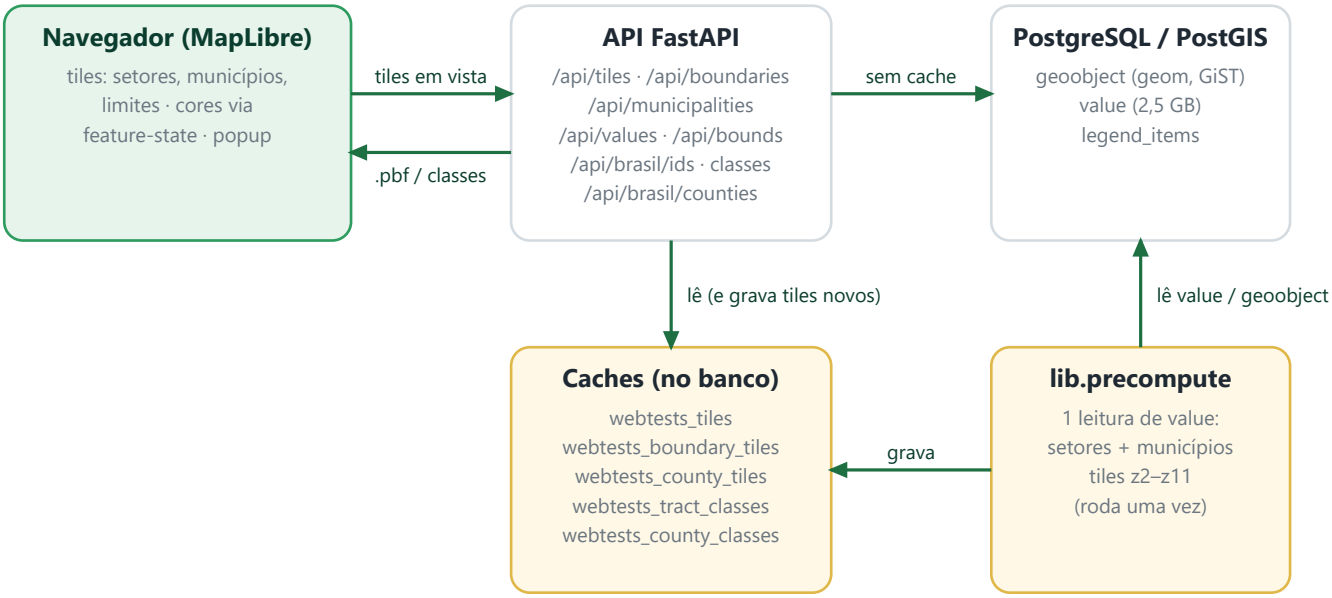
O aplicativo já mostrava estados pequenos e municípios rapidamente. Para o país, os números são de outra ordem:

Item	Tamanho	Consequência
Setores censitários (geoobjecttype 9)	468.097	GeoJSON de centenas de MB; inviável no navegador
Geometria dos setores	1,2 GB	Mesmo simplificada, muito pesada para enviar de uma vez

Item	Tamanho	Consequência
Tabela `value`	2,5 GB · 19 milhões de linhas	Organizada por setor (todos os indicadores juntos)
Ler 1 indicador para o Brasil	~200 s (disco frio)	As linhas do indicador estão espalhadas pela tabela inteira

A organização de `value` por setor é justamente o que torna os mapas de estado rápidos (todas as linhas de um setor ficam juntas no disco). O preço é que uma leitura "por indicador, no país todo" precisa tocar praticamente todas as páginas da tabela. Por isso o cálculo nacional tem de ser feito **antes**, e não a cada clique.

2. Visão geral da solução



Fluxo ao escolher Estado = Brasil

Não há botão "Exibir mapa": assim que setor estratégico, indicador e estado estão escolhidos, o mapa é carregado, e qualquer mudança nos campos o recarrega. Se uma carga ainda está em andamento quando o usuário muda um campo, ela é cancelada e descartada.

- O navegador baixa a lista de ids dos setores (uma vez por sessão, ~1 KB comprimida), a string de classes do indicador (~8 KB) e a legenda.
- Para cada setor com valor, aplica a classe como feature-state do MapLibre (468 mil chamadas, ~2 s no navegador).
- O MapLibre pede os tiles da área visível em `/api/tiles/{z}/{x}/{y}.pbf`. A API devolve o tile guardado em `webtests_tiles` ou, se ainda não existir, gera com PostGIS, guarda e devolve.
- A cor de cada polígono vem de uma expressão `match` sobre a classe, montada a partir das cores da legenda.
- Ao mesmo tempo, baixa as classes dos municípios (`/api/brasil/counties`, ~5.570 valores). Abaixo do zoom 8 o mapa desenha os municípios (tiles `/api/municipalities/…`); a partir do zoom 8, os setores (seção 7).

Com um estado ou município escolhido, o fluxo é o mesmo, mas em vez da string de classes o navegador baixa os valores da área (`/api/values`) e o retângulo para enquadrar o mapa (`/api/bounds`); os polígonos

vêm dos mesmos tiles (seção 4). Os limites de estados e municípios vêm de `/api/boundaries/{z}/{x}/{y}.pbf` (seção 5).

3. Geometria: tiles vetoriais com PostGIS

Cada tile é um quadrado do mapa num nível de zoom (padrão XYZ). A consulta de um tile:

- seleciona os setores que tocam o tile usando o novo índice espacial `geoobject_geom_gist`;
- simplifica a geometria com tolerância de meio pixel de tela no zoom do tile `ST_Simplify`;
- converte para coordenadas do tile com `ST_AsMVTGeom` (grade de 4096 unidades, corte nas bordas com margem de 64);
- gera o tile no formato Mapbox Vector Tile com `ST_AsMVT`. O id de cada feição é o `geoobject.id`; não há outros atributos (nome e município são buscados ao passar o mouse), o que deixa os tiles cerca de 40% menores.

Setores menores que 2 pixels

No zoom do país, um setor urbano ocupa menos de um pixel. Se fosse desenhado como polígono, as cidades sumiriam (ou virariam milhares de polígonos invisíveis). Nesses casos o tile desenha **um quadrado de 2×2 pixels por célula**, com a cor do maior setor da célula; ao aproximar o zoom os setores reais aparecem. Isso ainda distorcia o mapa do país, que por isso passou a mostrar municípios abaixo do zoom 8 (seção 7).

Os tiles não dependem do indicador: cada um é gerado uma única vez e guardado na tabela `webtests_tiles`. O navegador também os guarda por 1 dia (Cache-Control).

Tiles pré-calculados (zoom 2 a 11, área do Brasil)

Zoom	Tiles	Com dados	Total	Maior tile
2	2	2	0,2 MB	190 KB
3	4	3	0,7 MB	500 KB
4	9	7	1,9 MB	717 KB
5	20	15	3,4 MB	697 KB
6	64	40	5,3 MB	480 KB
7	225	124	9,0 MB	483 KB
8	870	442	15,9 MB	415 KB
9	3.420	1.632	25,5 MB	810 KB
10	13.447	6.237	37,2 MB	510 KB
11	52.864	24.323	54,2 MB	276 KB

Tiles sobre o oceano ou fora do Brasil ficam vazios e custam milissegundos. Os zooms 10 e 11 foram incluídos depois que aproximar sobre São Paulo levou ~30 s na primeira visita (o tile central do zoom 10 tem 15.504 setores, 3,6 s com o banco "quente"); geraram em 12,6 e 26,2 min. Os zooms 12 a 14 são gerados sob demanda na primeira visita (≤0,3 s por tile em São Paulo, poucas centenas de setores) e depois saem da tabela. O zoom máximo dos tiles é 14; acima dele o mapa amplia os tiles do zoom 14. No zoom 14 a geometria não é simplificada (seção 4).

4. Mapas de estado e município também em tiles

Antes, o mapa de um estado buscava o GeoJSON de todos os setores, simplificado polígono a polígono com `ST_SimplifyPreserveTopology` (tolerância 0,0005° ≈ 55 m). Isso tinha dois problemas:

- **Falhas entre setores** (“slivers”): cada polígono era simplificado sozinho, então a borda comum a dois vizinhos ficava diferente em cada um, abrindo frestas visíveis no zoom de rua.
- **Lento para estados grandes**: Minas Gerais (51.387 setores) levava **45,5 s**, quase todo o tempo gerando e transferindo a geometria.

Agora estados e municípios usam a **mesma camada de tiles** do mapa nacional:

- `/api/values` devolve, para cada setor da área, o valor e a cor da legenda; o navegador aplica ambos como feature-state `{color, value}`. Setores fora da área ficam transparentes.
- `/api/bounds` devolve o retângulo da área (`ST_Extent` dos municípios, algumas centenas de polígonos em vez de dezenas de milhares de setores), usado para enquadrar o mapa. Fica em cache no servidor.
- No zoom máximo (14) a tolerância de simplificação é zero: só resta o arredondamento para a grade do tile, que é igual para os dois lados de uma borda comum. Resultado: **nenhuma fresta**, mesmo ampliando até o zoom 16.
- O popup continua mostrando o valor exato do setor (ex.: 0,5000), que vem de `/api/values`.

Tempo de carga

Área	Setores	Antes (GeoJSON)	Agora (tiles)
Minas Gerais	51.387	45,5 s	3,1–4,0 s
Belo Horizonte	5.166	—	0,6 s

O endpoint `/api/geometry` (GeoJSON) foi removido.

5. Limites de estados e municípios

Como as linhas entre setores foram removidas, os limites administrativos vêm de uma segunda camada de tiles, `/api/boundaries/{z}/{x}/{y}.pbf`, gerada do mesmo jeito que a dos setores, mas com linhas (`ST_Boundary`) em vez de polígonos:

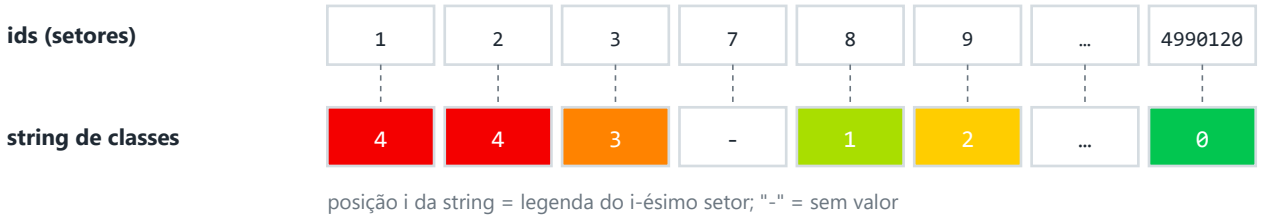
- camada `states`: limites dos 27 estados (geoobjecttype 4), em todos os zooms;
- camada `counties`: limites dos 5.570 municípios (geoobjecttype 1), a partir do zoom 6 (abaixo disso as linhas ficariam amontoadas);
- estilo: municípios em linhas brancas finas, estados em linhas escuras por cima dos setores.

A partir do zoom 11 os próprios setores também ganham um contorno branco fino. Ele não precisa de tiles novos: é uma camada de linhas sobre os mesmos tiles dos setores, só para os setores com cor (os de fora da área escolhida continuam invisíveis).

Os tiles de limites são pequenos (3–56 KB) e rápidos de gerar (0,1–0,6 s cada). Ficam na tabela `webtests_boundary_tiles`: 4.614 tiles dos zooms 2 a 9, **9 MB** no total (z6 0,7 MB · z7 1,3 MB · z8 2,2 MB · z9 3,7 MB). O custo para o navegador é desprezível comparado ao dos setores.

6. Cores: a “string de classes”

Para cada indicador, o servidor guarda um texto com um caractere por setor, na mesma ordem da lista de ids dos setores (crescente). O caractere é o índice da faixa da legenda (`0` a `9`) ou `-` quando o setor não tem valor:



posição i da string = legenda do i-ésimo setor; "-" = sem valor

- 468.097 caracteres ≈ 468 KB, que comprimem para **~8 KB** (há longas sequências iguais).
- A lista de ids vai codificada por diferenças (primeiro id, depois incrementos — quase todos iguais a 1): **~1 KB** comprimida.
- A string guarda a **versão** da lista de ids (hash). Se a lista mudar, o servidor recalcula e o navegador recarrega a lista.
- No navegador, cada classe vira `feature-state {c: classe}` do setor; a cor é escolhida pela expressão `['match', ['feature-state','c'], 0, cor0, 1, cor1, …]`.

Valor exato no popup

Como só a classe é pré-calculada, o popup do mapa nacional busca o valor exato do setor ao passar o mouse, junto com o nome: `/api/geoobject/{id}?indicator_id=` lê uma linha pelo índice (geoobject_id, indicator_id, year), em ~60–140 ms. A resposta fica em cache no navegador por setor e indicador; enquanto chega, o popup mostra a faixa da legenda.

Regra de classificação (correção de um erro)

As faixas da legenda são cadastradas como 0,00–0,19, 0,20–0,39, …, 0,80–1,00. A consulta antiga usava `minvalue ≤ valor < maxvalue`, que deixava de fora o valor **1,00** e os valores entre faixas (ex.: 0,195). No indicador "Balanço hídrico para agropecuária", **149.126 setores** com valor 1,00 ficavam invisíveis no mapa.

> A regra nova, usada no mapa nacional e também nos mapas de estado e município: **o valor recebe a faixa com o maior** `minvalue` **que seja ≤ valor**; valor ausente recebe a faixa sem limites ("Dado indisponível").

7. Municípios no zoom do país

Mesmo com os quadrados de 2 pixels, desenhar setores no zoom do país distorcia o mapa. Num pixel de vários quilômetros cabem centenas de setores urbanos, e a célula fica com a cor do **maior** setor. Na "Densidade populacional em setores censitários":

- **as cidades sumiam:** 50.282 setores "Muito alto" e 24.253 "Alto" quase não apareciam — o maior setor de cada célula costuma ser rural, de baixa densidade;
- **pontinhos cinza** espalhados: 1.101 setores sem dado ("Dado indisponível"), mas grandes (mediana 9,7 km², contra 0,13 km² dos setores em geral; 123 com mais de 100 km²).

Não é um problema de onde os dados ficam guardados: cada pixel só pode ter uma cor, então **sempre há uma agregação** — a questão é se ela é explícita ou escondida. A solução torna a agregação explícita:

- **Abaixo do zoom 8** (`COUNTY_FILL_MAX_ZOOM`), o mapa do Brasil mostra os **5.570 municípios**; **a partir do zoom 8**, os setores, cada um com a sua classe. A troca é automática ao aproximar ou afastar.
- A cor do município é a classe (mesma regra da legenda) da **média dos valores dos seus setores**, cada setor com o mesmo peso; setores sem valor ficam de fora, e o município só fica cinza se todos os seus setores estiverem sem dado.

- Os polígonos vêm de tiles próprios, `/api/municipalities/{z}/{x}/{y}.pbf` (zooms 2 a 7, ~4 MB, tabela `webtests_county_tiles`), simplificados com tolerância de 1/8 de pixel — com meio pixel, como nos setores, abriam frestas entre municípios vizinhos.
- As médias e classes são calculadas na mesma leitura de `value` que gera as strings de classes, e guardadas em `webtests_county_classes` (~33 KB por indicador). O popup do município mostra o nome e a média.

Média por setor × média ponderada pela área

A primeira versão ponderava cada setor pela área. Isso dá peso enorme a setores rurais quase vazios e repetia o problema: 5.551 dos 5.570 municípios ficavam "Muito baixo" na densidade. Os setores censitários são desenhados pelo IBGE com números de domicílios parecidos, então a média simples por setor se aproxima de uma média por **pessoa**:

Densidade populacional	Ponderada pela área	Média por setor (adotada)
São Paulo	0,29 · Baixo	0,72 · Alto
Rio de Janeiro	0,19 · Muito baixo	0,66 · Alto
Belo Horizonte	0,30 · Baixo	0,55 · Médio
Manaus	0,01 · Muito baixo	0,54 · Médio
Municípios Baixo ou acima	19	461

A média ponderada pela área responde "como é a maior parte do território", e pode fazer sentido para indicadores ligados à terra; nesse caso a regra pode passar a ser escolhida por indicador.

Nível da troca e teste

Um quadro no canto superior esquerdo do mapa mostra o zoom atual e o que está sendo desenhado ("ZOOM 7,4 · municípios"). No mapa do Brasil, um seletor muda o zoom da troca (4 a 12) na hora, para testar; ao recarregar a página volta o padrão do servidor (8). Entre os zooms 8 e ~10 os setores urbanos menores que 2 pixels ainda viram quadrados; a partir do 10–11 praticamente todos aparecem com a forma real.

As URLs dos tiles levam a versão de cada conjunto (`?v=`, `TILE_VERSIONS` em `lib/brasil.py`): ao regerar um conjunto, aumentar a versão faz o navegador descartar os tiles guardados (que ficam 1 dia no cache).

8. Pré-cálculo

Um único comando gera todos os caches:

```
python -m lib.precompute              # classes, tiles de setores, limites e municípios
python -m lib.precompute classes      # classes dos setores e dos municípios
python -m lib.precompute tiles 11     # só os tiles dos setores, até o zoom 11
python -m lib.precompute boundaries   # só os tiles de limites
python -m lib.precompute counties     # só os tiles de municípios (zooms 2–7)
docker compose exec app python -m lib.precompute   # dentro do Docker
```

- **Strings de classes:** uma leitura sequencial de `value` agrupada por indicador gera os 44 indicadores do setor censitário de uma vez — as strings dos setores e as médias dos municípios: **2 a 6 min no total** (conforme o disco esteja "quente"), contra ~200 s por indicador lendo um a um.
- **Tiles:** zooms 2 a 9 (e depois 10 e 11) sobre a área do Brasil, 4 em paralelo. Os zooms baixos são os mais lentos (cada tile cobre boa parte do país: ~8–9 min no zoom 2/3); do zoom 6 em diante, com a geometria já em memória no banco, cada nível leva de segundos a poucos minutos; o zoom 10 levou 12,6 min e o 11, 26,2 min. Os tiles de limites e de municípios são bem mais rápidos.

- O comando pula o que já existe: pode ser interrompido e executado de novo sem perder trabalho.
- Indicador sem string pré-calculada é calculado na primeira requisição (lento) e guardado; tile ausente é gerado na hora e guardado.

9. Desempenho medido

Operação	Tempo
Mapa do Brasil, primeira carga (ids + classes + legenda + tiles)	~5–8 s
String de classes de um indicador (do banco / da memória)	0,27 s / 0,03 s
Aplicar as cores de 468 mil setores no navegador	~2–3 s
Tile já guardado	~0,1 s
Tile novo, centro de São Paulo: z10 (15.504 setores) / z12	3,6 s / 0,3 s
Ler um indicador do país diretamente de value (sem pré-cálculo)	~200 s
Pré-cálculo das classes (setores + municípios, 44 indicadores)	2–6 min
Mapa de estado (valores, AC), com a nova regra de legenda	0,2–0,5 s
Mapa de Minas Gerais (51 mil setores): antes / agora	45,5 s / 3,1–4,0 s
Mapa de Belo Horizonte (5 mil setores)	0,6 s
Tile de limites novo (estados + municípios)	0,1–0,6 s

Durante o pré-cálculo dos tiles o banco fica ocupado; nesse período a primeira carga de um estado chegou a 69 s (a geometria do estado disputava disco com o pré-cálculo).

10. Objetos criados no banco (adaptabrasil)

Objeto	Para quê	Tamanho
`geoobject_geom_gist`	Índice espacial (GiST) em geoobject.geom, usado pela consulta de cada tile	20 MB
`webtests_tiles`	Tiles dos setores: (z, x, y) → bytes do tile (zooms 2–11, mais os gerados sob demanda)	160 MB
`webtests_boundary_tiles`	Tiles dos limites de estados e municípios (4.614 tiles)	9 MB
`webtests_tract_classes`	String de classes por indicador + versão da lista de ids	~2 MB
`webtests_county_tiles`	Tiles dos municípios, zooms 2–7 (mapa do país)	~4 MB
`webtests_county_classes`	Ids, classes e médias dos municípios por indicador	1,4 MB

As tabelas de cache são compartilhadas por todas as instâncias do aplicativo que usam o mesmo banco (a máquina local e o servidor 3.224.52.81): o trabalho é feito uma vez só.

Quando recalcular

- **Valores ou legendas mudaram:** `truncate adaptabrasil.webtests_tract_classes, adaptabrasil.webtests_county_classes;` e depois `python -m lib.precompute classes`. Reinicie o aplicativo (as strings também ficam em memória).
- **Geometria dos setores mudou:** `truncate adaptabrasil.webtests_tiles;` e depois `python -m lib.precompute tiles`. Se a lista de setores mudou, as strings de classes são recalculadas automaticamente (versão diferente).

- **Limites de estados ou municípios mudaram:** `truncate adaptabrasil.webtests_boundary_tiles, adaptabrasil.webtests_county_tiles;` e depois `python -m lib.precompute boundaries` e `counties`.
- Depois de regerar tiles, aumente a versão do conjunto em `TILE_VERSIONS` (`lib/brasil.py`), senão os navegadores mostram os tiles antigos por até 1 dia.

11. Limitações e próximos passos

- **Quadrados de 2 pixels entre os zooms 8 e ~10**, nos centros urbanos: os municípios resolveram o zoom do país, mas logo após a troca os setores muito pequenos ainda viram quadrados. O nível da troca pode subir (o seletor do mapa permite testar).
- **Mesma regra de agregação para todos os indicadores** (média por setor). Para indicadores ligados ao território pode valer a média ponderada pela área, escolhida por indicador.
- **~2–3 s para aplicar as cores** dos 468 mil setores no navegador (o mapa nacional leva ~5 s no total), mesmo que no zoom do país só os municípios apareçam. Pode cair aplicando as cores dos setores só ao chegar no zoom 8, ou só aos setores dos tiles carregados.
- **Contorno dos setores só a partir do zoom 11**: abaixo disso muitos setores são quadrados de 2 pixels e o contorno viraria ruído; as divisões visíveis são os limites de estados e municípios (seção 5).
- **PMTiles:** os zooms 12 a 14 ainda são gerados sob demanda. Os mesmos tiles (todos os zooms) podem ser exportados para um arquivo estático único (tippecanoe/PMTiles), servido sem consultar o banco — os dados mudam ~4 vezes por ano, então basta regerar a cada atualização.

12. Referência

Endpoints novos

Endpoint	Retorno
`GET /api/tiles/config`	Limites do Brasil, zoom mínimo/máximo, zoom da troca município/setor e versões dos tiles
`GET /api/brasil/ids`	Ids dos setores, crescentes, codificados por diferença (`version`, `deltas`)
`GET /api/brasil/classes?indicator_id=`	String de classes do indicador (`version`, `classes`)
`GET /api/tiles/{z}/{x}/{y}.pbf`	Tile vetorial dos setores (camada `tracts`, id = geoobject)
`GET /api/boundaries/{z}/{x}/{y}.pbf`	Tile de limites (camadas `states` e `counties`)
`GET /api/municipalities/{z}/{x}/{y}.pbf`	Tile dos polígonos dos municípios (camada `counties`, zooms 2–7)
`GET /api/brasil/counties?indicator_id=`	Ids, classes e médias dos municípios (`ids`, `classes`, `values`)
`GET /api/bounds?state_id=&county_id=`	Retângulo do estado ou município, para enquadrar o mapa
`GET /api/geoobject/{id}?indicator_id=`	Nome e município de um setor; com o indicador, também o valor (popup)

Removido: `GET /api/geometry` (GeoJSON dos mapas de estado).

Arquivos

Arquivo	Conteúdo
`lib/brasil.py`	Tiles de setores, limites e municípios (consultas MVT, cache), lista de ids, strings de classes, médias dos municípios, regra de legenda
`lib/precompute.py`	Pré-cálculo: `python -m lib.precompute`
`lib/db.py`	Nova regra de legenda nos mapas de estado; `fetch_bounds`; nome do setor para o popup
`main.py`	Endpoints novos
`static/app.js`	Opção Brasil; mapa carregado automaticamente; uma camada de tiles para todos os mapas; municípios no zoom do país; limites e contornos; cores por feature-state; popup; quadro do zoom

Com o Debug ligado, a requisição das classes mostra também a consulta SQL de um tile de exemplo, pronta para colar no DBeaver.